

Práctica 3

Metaheurísticas



Juan Antonio Mellado Arenas
Nicolás Sánchez Álvarez
Antonio Jesús Merlo Morales
Sergio Palacios López
Sergio García Atanasio

Universidad de Córdoba

Índice

1	Introducción	2
2	Enunciado de la práctica	2
3	Enfoques del problema	2
4	Representación y Función Objetivo	3
4.1	Representación	3
4.2	Función objetivo	5
5	Espacio de Búsqueda	6
6	Población inicial	7
6.1	Algoritmo Genético	7
6.2	Programación Genética	8
7	Selección de padres	8
8	Cruces entre individuos	9
8.1	Cruces del algoritmo genético	9
8.2	Cruces en programación genética	9
9	Mutación de individuos	9
9.1	Mutación del algoritmo genético	9
9.2	Mutación en programación genética	10
10	Reemplazo	10
10.1	Reemplazo en el algoritmo genético	10
10.2	Reemplazo en programación genética	11
11	Consideraciones extra	12
11.1	Consideraciones Algoritmo Genético	12
11.2	Consideraciones en programación genética	12
12	Análisis de resultados	14
12.1	Algoritmo genético	14
12.2	Regresión simbólica	19

1 Introducción

Documento sobre la realización de la tercera práctica de Metaheurísticas. Para esta, se ha implementado un algoritmo genético que resuelva el problema planteado por el profesor. Cualquier documentación adicional, así como los códigos fuente se encuentran en [el repositorio de la asignatura](#).

2 Enunciado de la práctica

Dados dos modelos A y B de clasificación que actúan a modo de caja negra, se pide explorar el espacio de entrada de estos. El objetivo es, en última instancia, llegar a aproximar su frontera de decisión (ya que comprender estas regiones es fundamental en numerosos contextos). Para resolver el problema hay total libertad, no obstante la metaheurística implementada ha de ser capaz de:

- Encontrar pares de puntos que pertenezcan a clases diferentes.
- Minimizar la distancia entre dichos puntos.
- Hallar la dispersión entre todos los puntos.

En este caso, hay únicamente dos tipos de puntos (un punto puede pertenecer a la clase 1 o a la clase 2) y el espacio sobre el que trabajaremos será el plano R^2 .

Para la resolución de este problema se ha implementado un algoritmo genético, pues se ha llegado al consenso de que es la opción más factible para la resolución de este problema, tanto por la facilidad de implementación del método como los buenos resultados que suelen ofrecer este tipo de algoritmos. Cada punto de este método basado en poblaciones se indica en su respectiva sección.

3 Enfoques del problema

Para este problema, nuestro grupo ha optado por dos líneas principales de trabajo, esto ha supuesto dividir toda la práctica en dos. El primer enfoque ha consistido en resolver el problema haciendo uso de un algoritmo genético híbrido, considerando un conjunto de puntos como un individuo, para, tras obtener varios puntos de ambas clases, poder esbozar la frontera de decisión.

El segundo enfoque, consiste en la utilización de programación genética y el uso de árboles para hallar una expresión matemática tal que, graficada, represente la frontera de decisión del modelo.

Así pues, el objetivo de la práctica, ha cambiado a no sólo resolver el problema propuesto por el profesor sino además hacerlo de dos maneras distintas, y, en última instancia compararlas, y hallar conclusiones de ambas metodologías.

Nota: Para cada sección del documento, se detalla el proceso seguido en ambas dos estrategias para abordar la práctica.

4 Representación y Función Objetivo

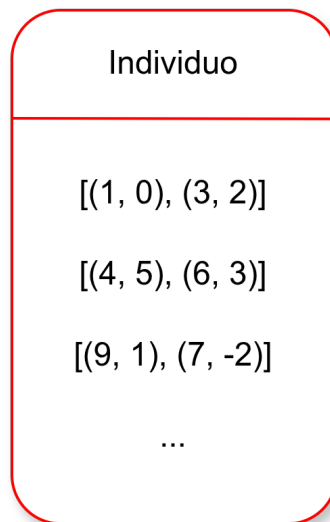
4.1 Representación

Para representar este problema, hay que entender que la solución final será una región de la aplicación R^2 , por lo que vendrá dada por una expresión (un polinomio por ejemplo) o bien la tendremos que esbozar e interpolar para unos puntos dados.

Así pues, las dos representaciones posibles son la expresión matemática que esboza la frontera, o bien un conjunto de puntos (un par de números enteros x e y) que puedan determinarnos de una manera aproximada la frontera que forma el modelo resultante.

4.1.1 Representación algoritmo genético

Para la realización de nuestro algoritmo genético, representaremos un individuo como un conjunto de pares de puntos del espacio en dos dimensiones, de un tamaño indefinido, pero que fijaremos nosotros previamente antes de comenzar nuestro algoritmo genético. Una representación visual abstracta puede ser la siguiente:



Además, para la implementación del individuo (no es la representación), se ha creado la clase `Individual`, con parámetros esenciales como el número de puntos del individuo (ha de ser par), los límites del espacio de búsqueda (posteriormente detallados), emparejamiento, valor de fitness y la clase a la que pertenece cada punto. Por último los puntos se almacenan en una matriz de $n \times 2$ donde cada fila representa un punto, a efectos prácticos es como una lista de puntos en orden ya que así la recorreremos. Dicho esto, si el tamaño de los individuos es de 20, tendremos una lista de 20 puntos que están emparejados entre sí.

El vector "pairs" indica la posición de cada punto en el vector de puntos. Es decir, un elemento del vector de pares podría ser (4, 8), que indica que los puntos asociados a ese par son: (points[4], points[8])

```

1  class Individual:
2      def __init__(self, numPoints: int = 20,
3                  limits: Tuple[float, float] = (-2.0, -2.0),
4                  model: modelo.BlackBoxModel = None):
5          self.numPoints = numPoints
6          self.limits = limits
7          self.model = model
8
9          self.points = np.random.uniform(limits[0], limits[1], (numPoints, 2))
10         self.classes = None
11         self.fitness = None
12         self.pairs = []
13         self.components = {}

```

Listing 1: Constructor de la clase Individuo

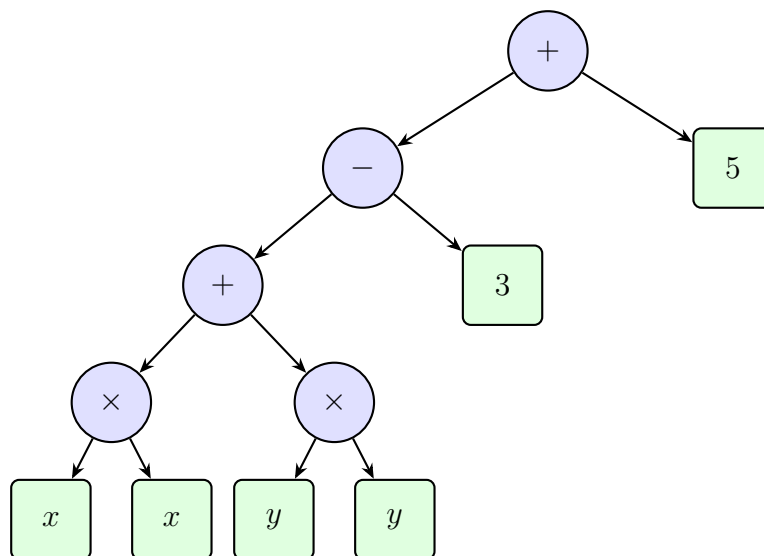
Nota: Components son los componentes que se usarán para calcular el fitness, ya que depende la disposición de todos los puntos del individuo (dispersión, tipos de puntos...), esto se detalla a continuación.

4.1.2 Representación programación genética

La representación del individuo en el algoritmo de programación genética se ha utilizado un AST de profundidad variable. Los operadores (o nodos intermedios) posibles son **suma**, **resta** y **producto** (`add()`, `sub()`, `mul()`). Los números (reales en $[-5, 5]$) o variables x y y se alojan en las hojas.

La solución última se representa como la expresión de una curva y se obtiene recorriendo el árbol de forma infija, e igualando la expresión resultante a 0.

Un individuo que represente la expresión $x * x + y * y - 3 + 5$ se puede visualizar de esta forma:



4.2 Función objetivo

4.2.1 Función objetivo algoritmo genético

Para poder evaluar en qué medida un individuo (conjunto de puntos) soluciona el problema, tendremos que tener en cuenta las siguientes características que un individuo puede tener.

- **Distancia media de los puntos de los pares:** Los puntos se emparejan, y asumimos que un par de puntos es mejor si la distancia entre sus dos puntos es mínima. Esto es porque, si la distancia entre una pareja de puntos es pequeña, el error a la hora de estimar la frontera es mucho menor. Para evaluar el individuo, se aplica la distancia euclidiana en el plano en cada par de puntos, y la media resulta en este componente.
- **Dispersión:** O distancia entre los pares del individuo. Para calcular este componente de la función objetivo, se halla la distancia mínima entre todos los pares. Si por ejemplo tuviéramos cuatro puntos (dos pares), si están cerca significa que al menos un punto de un par está cerca de al menos un punto del otro par, por eso se hallan las distancias entre puntos de diferentes pares y se obtiene la más pequeña de todas. Buscamos que la dispersión sea máxima, ya que queremos la mayor variedad de parejas de puntos para hacer la mejor estimación posible con ellos.
- **Variedad:** Naturalmente, de nada sirve obtener puntos en los que todos sean del mismo tipo, por lo que primeramente se comprueba cuántos puntos hay de cada clase. En adición, se mide para cada par si ambos puntos son del mismo tipo o no. Por ejemplo si trabajamos con 10 puntos y hay 5 de cada tipo, el componente de variedad será perfecto, pero si al emparejar esos puntos no se emparejan tal que cada uno esté con el del tipo opuesto (esto ocurrirá en muchos individuos), se añade un componente de misma clase, que evalúa cuantos pares hay con dos puntos distintos respecto del total, siendo en este caso 5 pares, y según si haya más o menos se penalizará más al individuo o menos.

Una vez indicados todos estos valores, la función de fitness se calcula multiplicando cada componente por un valor de peso (para controlar cuáles de éstas características son más relevantes). Con estos valores tan sólo queda sumar la distancia entre puntos de un par (cuánto menos mejor), sumar las penalizaciones de variedad y pares del mismo tipo (si es 0 porque hay equilibrio no suma nada, y si es más, se suma), y por último, se resta la dispersión. Por lo tanto, nuestro objetivo será minimizar la función de fitness, ya que la distancia interpunto queremos que sea 0 (la sumamos) y la intrapunto queremos que sea la máxima posible (la restamos)

4.2.2 Función objetivo programación genética

Para la regresión simbólica, la función objetivo evalúa cómo de cerca la expresión matemática representada por un individuo (árbol AST) se aproxima a la frontera de decisión del modelo caja negra. Se definen los siguientes componentes:

- **Puntos de bisección:** Se genera una nube de puntos que pertenecen a la frontera real. Cada punto se obtiene promediando dos puntos de clase distinta separados por una distancia intrapar

menor a un umbral (0.15 para el modelo B, 0.4 para el modelo A). Estos puntos representan la verdad fundamental que la expresión debe aproximar.

- **Puntos de contraste dinámicos:** Para evitar que la función se anule en regiones alejadas de la frontera, se añaden puntos de contraste calculados a partir de la extensión de la nube de bisección. Normalmente se toman cuatro puntos en las esquinas del rectángulo que engloba la nube más un margen del 20%, y un punto central. Se exige que en estos puntos el valor absoluto de la función sea significativamente distinto de cero.
- **Penalización por tamaño:** Para favorecer expresiones parsimoniosas, se añade un término proporcional al número de nodos del árbol (penalización de 0.07 por nodo).

La función de fitness se define como:

$$\text{fitness} = 1000 \cdot \frac{1}{N_p} \sum_{(x,y) \in P} f(x,y)^2 + \frac{1}{N_c} \sum_{(x_c,y_c) \in C} \frac{1}{f(x_c,y_c)^2} + 0.07 \cdot |\text{individuo}|$$

donde P es el conjunto de puntos de bisección, C el conjunto de puntos de contraste, f la función expresada por el árbol, y $|\text{individuo}|$ el número de nodos del árbol.

Se penalizan con ∞ aquellos individuos que:

- No contengan las variables x e y en su expresión.
- Generen desbordamiento numérico o divisiones por cero durante la evaluación.
- Produzcan un rango de salida demasiado pequeño (casi constante).
- En algún punto de contraste el valor absoluto sea menor a 0.1.

El objetivo es minimizar este fitness, de modo que la expresión se aproxime a cero sobre la frontera real y se aleje de cero en el resto del espacio, todo ello con la máxima simplicidad estructural.

5 Espacio de Búsqueda

El problema principal que nos encontramos en este apartado es que el tamaño del conjunto R2 es infinito, entonces aunque quisiéramos, no podríamos hacer una búsqueda exhaustiva.

La idea general será, por lo tanto, ir probando espacios de diferentes límites y tamaños hasta encontrar al menos un punto que pertenezca a otra clase con respecto a los demás puntos. Además, también hay que considerar que la frontera pueda formar una curva cerrada, así que ampliaremos el espacio en aquellos casos que no se sepa del todo si la frontera será cerrada o abierta.

En nuestro caso particular, hemos tenido la suerte que no ha habido que probar muchos espacios de búsqueda en el modelo B, pues podemos obtener puntos de diferente clase en un espacio delimitado por $[-1.0, 1.0]$ en ambos ejes. Por el contrario, para el modelo A hemos tenido que ir aumentando poco a poco el espacio de búsqueda hasta encontrar uno que se ajuste al tamaño real de la frontera, por ejemplo vemos que si dejamos el espacio de búsqueda fijo, la frontera no queda cerrada correctamente.

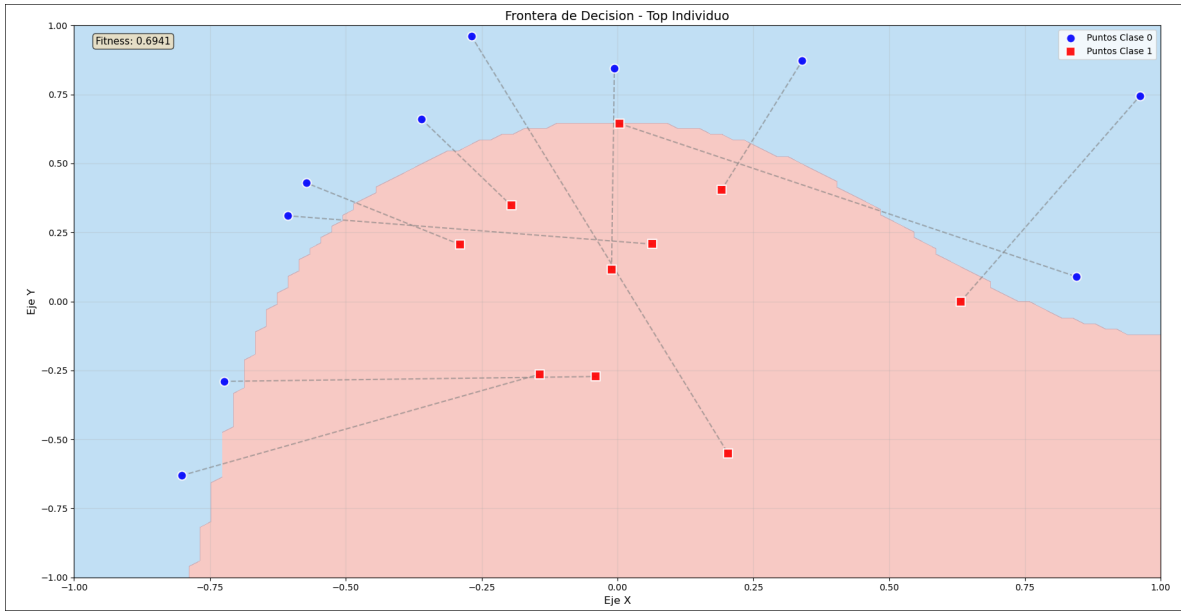


Figure 1: Puntos de un individuo en intervalo $[-1.0, 1.0]$ para el Modelo B

Nota: Esto se comprueba en el resultado final del algoritmo genético, pero el procedimiento de este será explicado igualmente.

6 Población inicial

6.1 Algoritmo Genético

El primer paso para realizar el algoritmo genético consiste en la creación de una población inicial que modificaremos después de varias generaciones y obtener mejores individuos. Para el algoritmo se ha implementado una generación de población inicial estocástica. Esta consiste en crear individuos aleatorios, dentro del espacio de búsqueda para que los individuos tengan sentido en la representación del problema. Esta estrategia es la de menos coste y complejidad, y ofrece precisamente un balance entre diversidad y coste computacional.

La principal debilidad de este método es que al no haber un control de los individuos resultantes, cabe la posibilidad de que surgan dentro de un individuo muchos puntos parecidos, y dentro de la población a su vez muchos individuos parecidos. Esto, además de ser poco probable que la mayoría de puntos sean similares, se abordará posteriormente con algunas técnicas que influirán más en la diversidad del algoritmo.

Un matiz que se ha tenido en cuenta es que, a la hora de emparejar los puntos aleatorios que se han generado al crear el individuo, se preferirá de primera mano emparejar aquellos puntos que pertenezcan a una clase diferente, y el resto de puntos que sea de la misma clase emparejarlos aleatoriamente entre sí. De esta forma, ya empezamos de primera mano con individuos que tienen un fitness más aceptable que si se emparejasen todos los puntos de manera aleatoria, con muy poco coste computacional añadido.

En general, aplicando estos principios, obtendremos unos resultados constatemente buenos entre

ejecuciones.

6.2 Programación Genética

Para la programación genética, la población inicial se genera de forma completamente aleatoria siguiendo un enfoque de crecimiento controlado (equivalente al método *ramped half-and-half* simplificado). Se crean un total de **500 individuos**, cada uno de los cuales es un árbol de sintaxis abstracta (AST) que representa una expresión matemática. La profundidad de cada árbol se elige aleatoriamente entre **1 y 3** (ambos inclusive), lo que garantiza expresiones inicialmente sencillas pero con diversidad estructural.

Los nodos internos se seleccionan del conjunto de operadores {suma, resta, multiplicación} (`add`, `sub`, `mul`), mientras que las hojas pueden ser:

- La variable independiente x ,
- La variable independiente y ,
- Constantes numéricas reales generadas aleatoriamente en el rango $[-5, 5]$.

La generación se realiza mediante el método de “crecimiento completo” para profundidades inferiores a la máxima, y “crecimiento parcial” en el último nivel para asegurar que todos los árboles sean sintácticamente válidos y tengan hojas en sus ramas terminales.

No se aplica ninguna poda ni corrección inicial sobre los individuos, pues se busca preservar la máxima diversidad en la primera generación. La única restricción es que la raíz del árbol sea siempre un operador (no un terminal), para evitar expresiones triviales como una única constante o variable.

Esta estrategia de inicialización es de bajo coste computacional y permite explorar un amplio abanico de formas funcionales desde las primeras generaciones, complementando la posterior selección, cruce y mutación descritas en secciones posteriores.

7 Selección de padres

Para elegir los padres de la población, se sigue la selección por torneo, en la cual se escogen algunos individuos, y aquel con mejor fitness (menor valor de función de fitness), es escogido como padre, el proceso se repite y para cada padre se efectúa un torneo. Esta técnica es más costosa que seleccionar los padres aleatoriamente, no obstante, garantizamos que los hijos que se generen sean lo mejores posibles para intensificar a la hora de buscar nuestra mejor solución.

Se ha definido el tamaño del torneo como 3, ya que si fuese mucho mayor, es posible que la presión selectiva sea demasiado elevada corriendo el riesgo de que siempre se escojan los mismos padres y por lo tanto haciendo que nuestra población sea muy parecida.

Escogeremos además que las selecciones para participar en el torneo son con reposición, pudiendo aparecer varias veces el mismo individuo.

8 Cruces entre individuos

8.1 Cruces del algoritmo genético

El objetivo del cruce entre individuos es obtener los hijos, que luego serán los candidatos para el reemplazamiento de la población. Para este caso, se ha implementado un operador de cruce uniforme entre pares. El cruce combina pares de los diferentes padres, respetando su estructura. Para cada par de puntos del individuo hijo, se decide aleatoriamente de que padre se obtiene, aunque, se garantiza que haya de ambos dos progenitores sí o sí. La combinación de pares contraria al hijo creado resulta en el segundo hijo del cruce.

Es necesario que se intercambien los pares entre sí, pues si intercambiamos únicamente puntos sin tener en cuenta su contraparte, obtendremos un hijo que no se parece en nada a ninguno de sus padres y además lo más probable es que el hijo resultante sea peor que de la otra forma.

Tras cruzar ambos padres, se vuelve a evaluar el individuo, obteniendo las clases de sus puntos y pares y con la función objetivo. Se ha escogido este método porque al respetar los pares como indivisibles, se mantiene un equilibrio entre la coherencia de cada par de puntos y la dispersión general del individuo. Otra de las ventajas es que no existe un orden implícito en nuestra representación, por lo que esta estrategia no resulta apenas costosa computacionalmente.

8.2 Cruces en programación genética

Al trabajar con estructuras de árboles de sintaxis abstracta (AST) en lugar de representaciones vectoriales clásicas, los operadores de variación deben adaptarse a la naturaleza jerárquica de las soluciones. Para este modelo, se ha empleado el cruce en un punto, implementado a través de la función `gp.cxOnePoint` de la librería DEAP.

Conceptualmente, este operador selecciona un nodo de corte de forma aleatoria en el árbol del primer progenitor y realiza la misma acción en el árbol del segundo progenitor. A partir de estos dos nodos, el algoritmo intercambia los subárboles (o ramas) completos entre ambos padres. El resultado de este proceso son dos nuevos individuos descendientes que heredan e integran diferentes fragmentos de las expresiones matemáticas originales, permitiendo explorar nuevas combinaciones y formas geométricas en la frontera de decisión sin destruir la validez sintáctica de la ecuación.

9 Mutación de individuos

9.1 Mutación del algoritmo genético

Antes de proseguir en el algoritmo, algunos individuos mutan bajo un operador que selecciona ciertos individuos, y altera genes de este (en este caso altera puntos). Para cada mutación, el operador modifica levemente valores en un eje, o en ambos, de puntos de ese individuo. Esto se hace para promover más diversidad y traer puntos nuevos que mejoren las generaciones.

No obstante, este operador de mutación, es poco agresivo, ya que por la índole del problema la diversidad está garantizada, y porque se ha implementado un segundo operador que cumple otra

función más.

Cada un número de generaciones, el individuo con mejor fitness pasa por un proceso de reducción de distancia para sus pares de puntos. Este operador recorre todos los puntos del individuo hallando la recta que forma ese par de puntos, y juntando ambos en el plano de R2. De esta manera los pares de puntos se encontrarán en la frontera de decisión, permitiendo casi intuir la forma que esta tendrá, y posteriormente incluyendo esos pares en la población.

Este operador es muy agresivo, complejo y costoso, pero permite obtener soluciones de altísima calidad.

9.2 Mutación en programación genética

Para la mutación se han implementado **2 tipos**:

- **Mutación Uniforme**: Este tipo de mutación, se selecciona **un nodo al azar** del individuo a mutar desde a partir del cual hasta el final del árbol será el **subarbol escogido**. Una vez con ello, se genera un subarbol completamente **aleatorio** de una profundidad entre 0 y 2, el cual se **sustituirá** por el seleccionado en el Padre a mutar. Esta configuración da lugar tanto a la posibilidad de que se "elimine" el subarbol seleccionado (al insertar un subarbol de profundidad 0), como la seguridad de que al sustituir por un subarbol de profundidad ≥ 0 , no vamos a aumentar demasiado la complejidad del mismo, pero sí aportarle una muy buena **diversidad**.
- **Mutación de contracción (poda)**: Con este tipo de mutación, además de aumentar la diversidad, **disminuimos drásticamente la complejidad** y la profundidad de los individuos **demasiado grandes**. En este caso, al igual que en el anterior, busca un nodo de forma **aleatoria** en el padre a mutar, pero con la **restricción de que este sea una función** (multiplicación, suma, resta...), no un terminal (como x o un número). Una vez escogido dicho nodo, selecciona al azar uno de sus hijos, una vez hecho esto, elimina todo el bloque del padre y lo cambia por el hijo seleccionado. Esto provoca que si ese nodo tenía una gran cantidad de hijos, siendo estos además de una alta complejidad, los elimine haciendo que la complejidad del individuo disminuya inmensamente.

Finalmente, la escogida para usar posteriormente en la obtención de las métricas y el uso del algoritmo en general, fue la mutación uniforme, ya que aporta una gran diversidad que ayuda enormemente a la obtención de mejores individuos y la mejora de las soluciones. Además, observamos que la mutación de poda se comportaba de forma muy agresiva, reduciendo demasiado la complejidad de los individuos, haciendo que obtuviesemos de forma demasiado frecuente funciones como líneas rectas y similar.

10 Reemplazo

10.1 Reemplazo en el algoritmo genético

El reemplazo de individuos en este algoritmo utiliza un modelo de Elitismo de la siguiente forma:

- **Fase de Elitismo:** Antes de comenzar a generar nueva descendencia, se ordena la población actual de mejor a peor según su *fitness*. Los individuos más aptos, cuya cantidad está definida por el parámetro `ELITE_SIZE` (valor: 2), se copian directamente a la nueva generación. Estos individuos pasan intactos, asegurando que la mejor solución del algoritmo nunca se pierda por culpa de un cruce o mutación desfavorable.
- **Cruce y selección:** Una vez que los mejores están elegidos tenemos que rellenar los 48 huecos restantes, para hacerlo escogemos a tres individuos al azar, los dos mejores tendrán la posibilidad de cruzarse `CROSSOVER_PROB` (valor: 0.85) una vez hayan sido cruzados o clonados se van metiendo en la nueva generación hasta ocupar los huecos
- **Transición final:** Cuando la nueva población está completamente llena guardamos y pasamos a mutar, fase en la que solo se eligen a los individuos nuevos, dejando intactos a los de la élite.

10.2 Reemplazo en programación genética

En la programación genética implementada para este problema, el reemplazo de individuos sigue un modelo **generacional** sin elitismo explícito, gestionado mediante la función `eaSimple` de la librería DEAP. El proceso se describe a continuación:

- **Tamaño de población constante:** Se mantienen 500 individuos por generación.
- **Generación de descendencia:** En cada generación, se crea un conjunto de hijos a partir de la población actual. Para ello:
 - Se seleccionan padres mediante torneo (tamaño 3) con reposición.
 - Se aplica cruce en un punto (`cxOnePoint`) con probabilidad `cxbp = 0.7`.
 - Se aplica mutación uniforme (`mutUniform`) con probabilidad `mutpb = 0.3`.
 - El número total de hijos generados es igual al tamaño de la población (500), por lo que cada nueva generación se compone íntegramente de descendientes.
- **Pérdida de los padres:** Los individuos de la generación anterior desaparecen completamente, a menos que sean seleccionados como padres y sus variantes pasen a la siguiente generación. **No se copia ningún individuo directamente** (sin elitismo).
- **Hall of Fame:** Para no perder la mejor solución encontrada durante toda la ejecución, se utiliza un `HallOfFame` de tamaño 1. Este almacena el mejor individuo global, pero **no lo reintroduce automáticamente** en la población. Al final del algoritmo, se recupera de él la mejor ecuación.

Este enfoque generacional simple favorece la exploración del espacio de búsqueda, evitando la convergencia prematura. Sin embargo, puede provocar la pérdida temporal de buenas soluciones si no generan descendencia exitosa. Para mitigarlo, el `HallOfFame` actúa como una memoria externa que garantiza que la mejor expresión matemática encontrada no se descarte al final del proceso.

A modo de comparación, el algoritmo genético híbrido explicado anteriormente emplea un reemplazo con elitismo fuerte (copia directa de los dos mejores individuos), lo que intensifica la explotación

de las mejores soluciones. En la programación genética, se prioriza la diversidad frente a la presión selectiva, dado que las expresiones simbólicas pueden beneficiarse de una mayor variabilidad estructural.

11 Consideraciones extra

11.1 Consideraciones Algoritmo Genético

Es posible que algunas de estas consideraciones hayan sido mencionadas en otros apartados, pero recapitulando, estas son los cambios más relevantes frente a una metodología tradicional en un algoritmo genético:

- **Minimización de la distancia entre puntos de un mismo par:** Se tuvo en cuenta finalmente ya que al algoritmo le costaba mucho trabajo encontrar puntos aleatoriamente que estuviesen cerca entre sí, por lo que se introdució este procedimiento para que la aproximación de la frontera sea lo más fiel a la original posible.

Para ello, se genera una recta entre ambos puntos que pertenecen a su mismo par. Una vez obtenida la recta a través de la pendiente, podemos aplicar una búsqueda local para cada punto, que encuentre el punto más cercano a su contrario y que pueda pertenecer a su clase original (que no cambie de clase). Después, se repite el mismo proceso para el otro punto, pero en dirección contraria. De esta forma, se obtienen los puntos lo más cercanos de uno al otro cuya separación es la frontera.

El único inconveniente es que no funciona para puntos de la misma clase, por ello luego no son relevantes para realizar la estimación, aunque idealmente el mejor individuo tendría todos los pares con puntos de diferente clase.

Este método será usado cada 5 generaciones sobre el mejor individuo, ya que es una operación costosa para realizarla sobre todos los individuos de la población.

- **Generación de la frontera estimada:** La frontera estimada es calculada mediante el punto medio generado entre los puntos pertenecientes a un mismo par. El único requisito de estos puntos fronterizos ficticios es que tienen que ser generados por un par con puntos de diferente clase, ya que de otra forma no es lo suficientemente representativo como para formar parte de la solución final.

Una vez obtenidos, se dibuja la frontera uniéndolos los puntos que sean más cercanos entre ellos para intentar formar una forma cerrada.

11.2 Consideraciones en programación genética

A continuación se enumeran las consideraciones más relevantes adoptadas en la implementación de la programación genética, que se apartan de un enfoque estándar para adaptarse a las particularidades del problema de aproximación de fronteras de decisión.

- **Manejo de individuos inviables:** Se asigna un fitness ∞ a aquellos individuos que:

- No contengan las variables x e y en su representación en cadena.
- Durante la evaluación provoquen desbordamiento numérico (`OverflowError`) o división por cero.
- Produzcan un rango de salida (máximo - mínimo) inferior a 10^{-5} (función prácticamente constante).
- En algún punto de contraste se cumpla $|f(x_c, y_c)| < 0.1$.

Esta estrategia filtra expresiones no válidas sin detener la evolución.

- **Penalización por complejidad estructural:** Para favorecer la parsimonia (expresiones más simples y legibles), se añade al fitness un término de $0.07 \cdot (\text{número de nodos del árbol})$. Este valor se ajustó empíricamente para que no domine sobre los errores de aproximación pero sí penalice árboles excesivamente grandes.
- **Uso de Hall of Fame sin elitismo en el reemplazo:** A diferencia del algoritmo genético (Sección 11.1), en la programación genética no se copian automáticamente los mejores individuos a la siguiente generación. En su lugar, se emplea un `HallOfFame` de tamaño 1 que almacena la mejor expresión encontrada a lo largo de toda la ejecución, pero no la reintroduce en la población. Esto permite una mayor exploración al evitar la presión selectiva excesiva, aunque se sacrifica la convergencia asegurada. Al final, la ecuación final se recupera del `HallOfFame`.
- **Transformación de la ecuación final a formato legible:** Mediante la librería `sympy` se simplifica la expresión del mejor individuo y se traduce a una ecuación igualada a cero, lista para ser utilizada en herramientas externas como GeoGebra. Un ejemplo de ello se muestra en el archivo `bestBecuation.txt`, donde se traduce una expresión compleja a la forma:

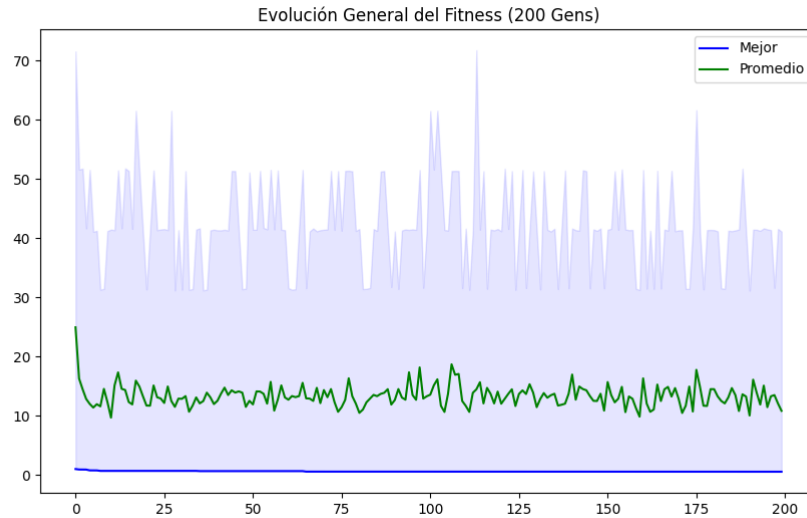
$$-3.64997 x^6 y^2 - 0.437395 x^2 - 0.437395 y^2 + 0.201545 = 0$$

12 Análisis de resultados

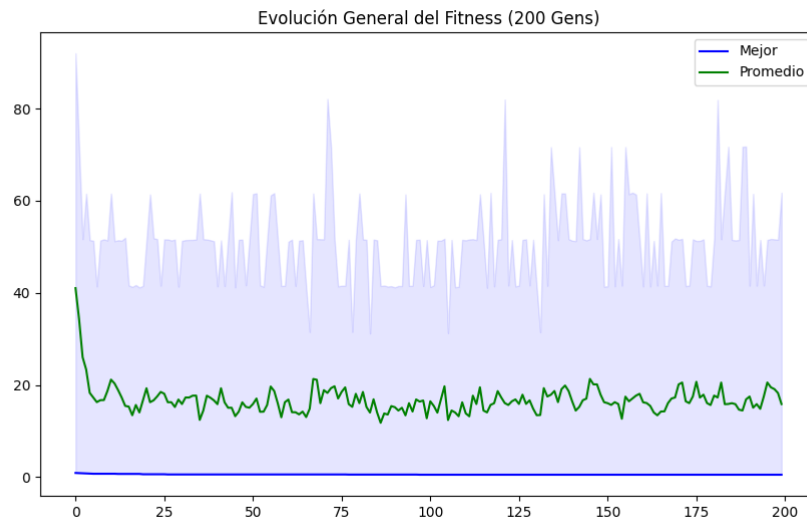
12.1 Algoritmo genético

12.1.1 Evolución del fitness

Al observar la gráfica de la evolución general para el modelo A con un poco de esfuerzo podemos apreciar como el algoritmo encuentra rápidamente buenas soluciones y se estanca, el fitness final es 0.67, mientras que el fitness promedio de la población es aproximadamente 11.

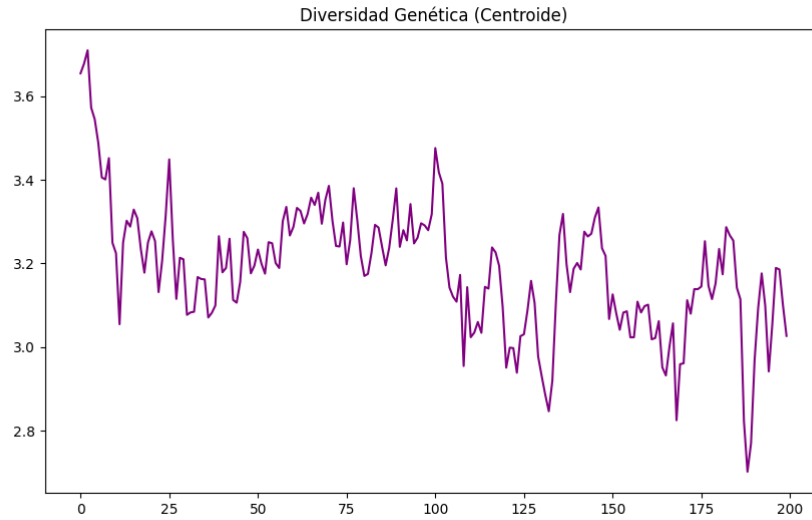


Por otro lado el modelo B podemos apreciar que comienza con una peor población general pero al llegar a la generación pero también converge llegando a un fitness mínimo de 0.9

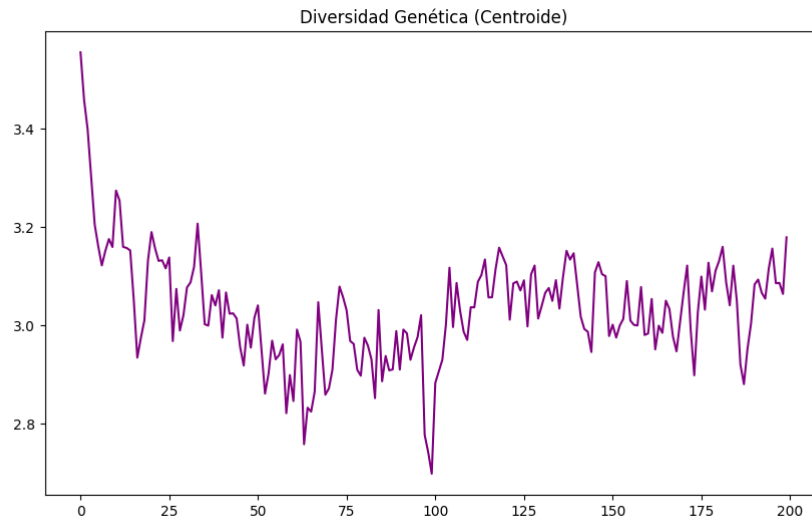


12.1.2 Diversidad genética

Para el modelo A la gráfica muestra que la distancia promedio de los individuos al centroide fluctúa constantemente. Tras una pequeña caída inicial ya que se descartan los soluciones iniciales peores, la curva sube y baja de forma periódica en lugar de caer hacia cero lo que demuestra que la población nunca llega a colapsar.

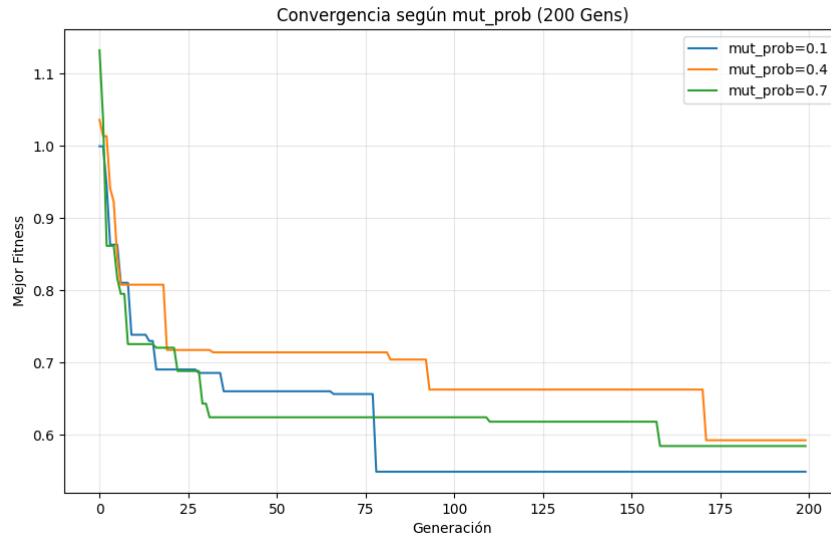


Para el modelo B podemos ver como se mantiene siempre alta y variante. Esto confirma que la población no colapsa en clones idénticos; los operadores de cruce y mutación logran inyectar continuamente nuevo material genético.

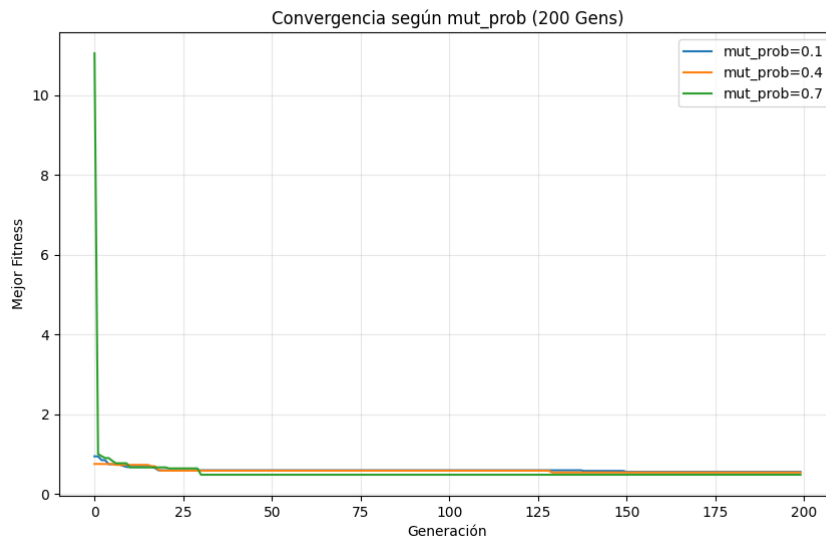


12.1.3 Probabilidad y tasa de mutación

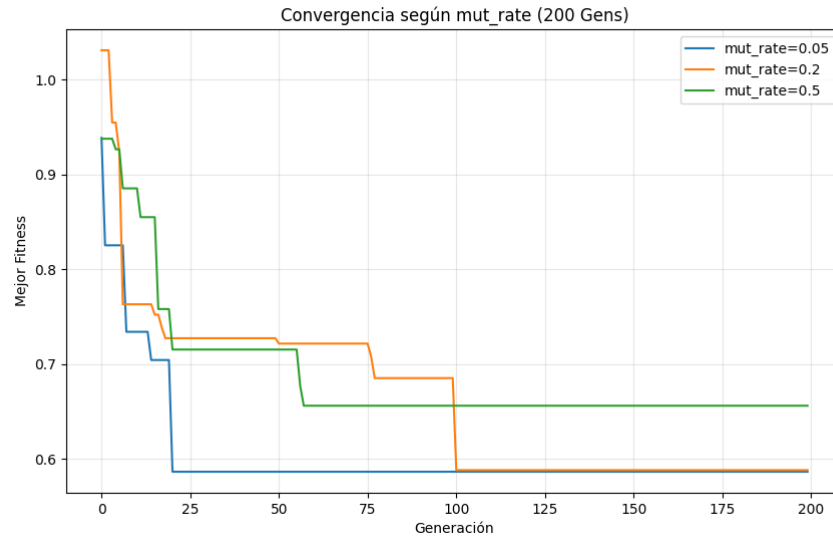
El estudio de la mutación se divide en dos parámetros, la probabilidad de mutación `mut_prob`, por la cual podemos apreciar como para el modelo un valor de 0.1 consigue el mejor fitness para valores de `mut_rate` bajos (0.2)



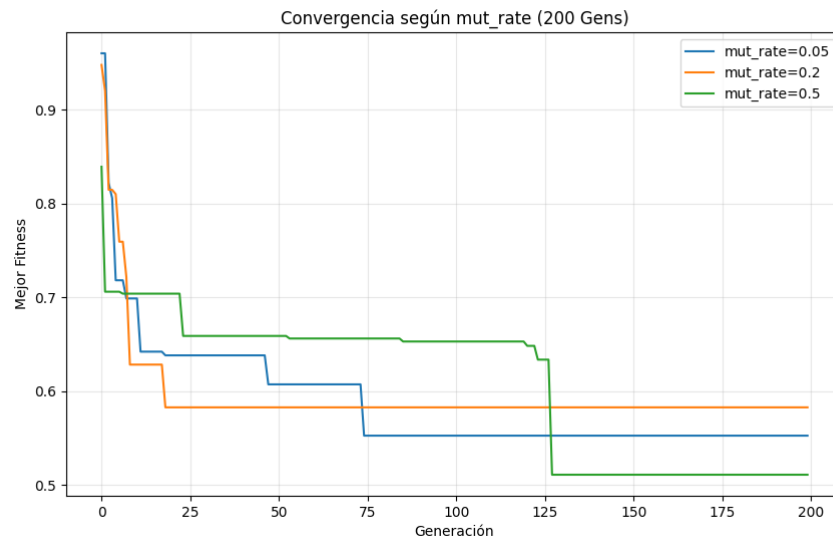
Sin embargo para el modelo B la relación es menos distante, pese a esto ganaría mayor probabilidad de mutación para valores de mut.rate bajos



Por otro lado tenemos la intensidad con la que se modifican sus genes el mut.rate lo que hace que se diferencien más entre generaciones apreciando un claro vencedor del valor 0.05 a lo largo de las generaciones para el modelo A

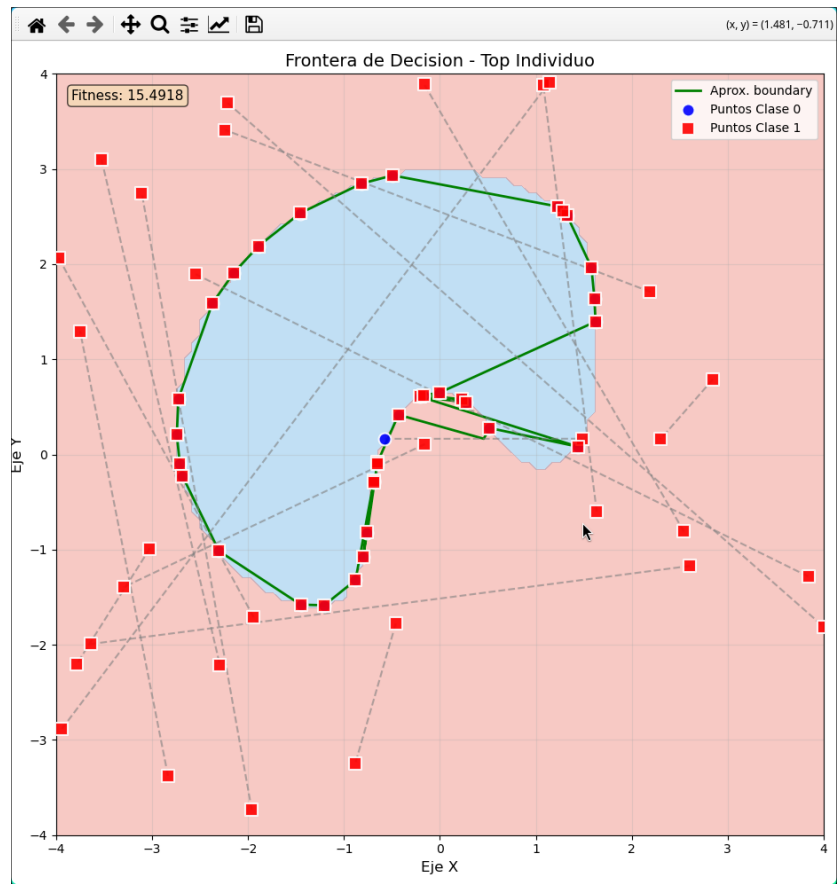


Y del valor 0.5 para el modelo B

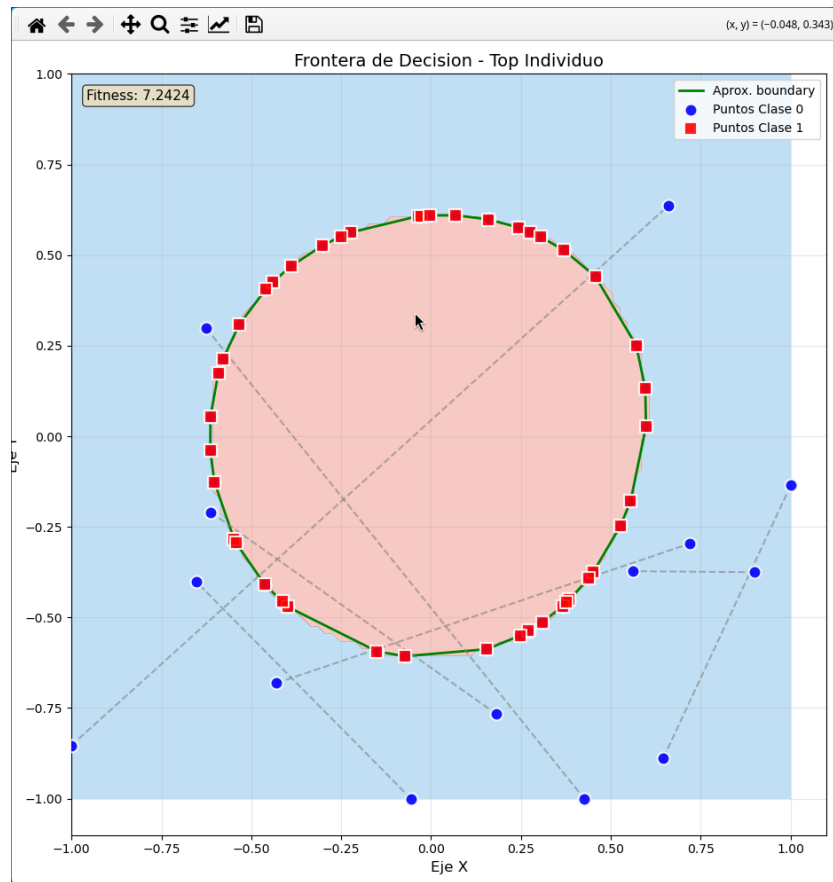


12.1.4 Análisis de la frontera

Por último respecto al estudio del algoritmo genético observamos como quedarían las fronteras, tanto para el modelo A



Como para el modelo B



Nota: Aunque parezca que sólo haya puntos de clase 1 en la frontera estimada, realmente son una pareja de puntos pero con la clase 1 prácticamente superpuesta sobre la clase 2. Entre ambos puntos existe un punto medio que conforma la frontera, pero no se ve en el gráfico.

12.2 Regresión simbólica

El análisis de la programación genética aplicada a la identificación de las fronteras de decisión revela comportamientos distintos según la complejidad geométrica de cada modelo de caja negra:

12.2.1 Análisis del Modelo A (Parábola)

Para el Modelo A, el algoritmo debe capturar una frontera con apertura infinita. Los resultados muestran las siguientes tendencias:

- **Eficiencia Temporal:** Los tiempos de ejecución (ver Fig. 2) se mantienen estables, reflejando la carga computacional de evaluar 35 puntos de bisección a lo largo de 1000 generaciones.
- **Convergencia de Fitness:** Se observa una reducción drástica del error en las primeras etapas (ver Fig. 3), logrando una aproximación precisa de la curva parabólica gracias al uso del *Hall of Fame*.
- **Estabilidad Numérica:** La desviación típica (ver Fig. 4) presenta picos puntuales debido a la naturaleza polinomial de las ecuaciones, donde pequeñas variaciones en los coeficientes pueden disparar el error antes de ser filtradas por la selección natural.

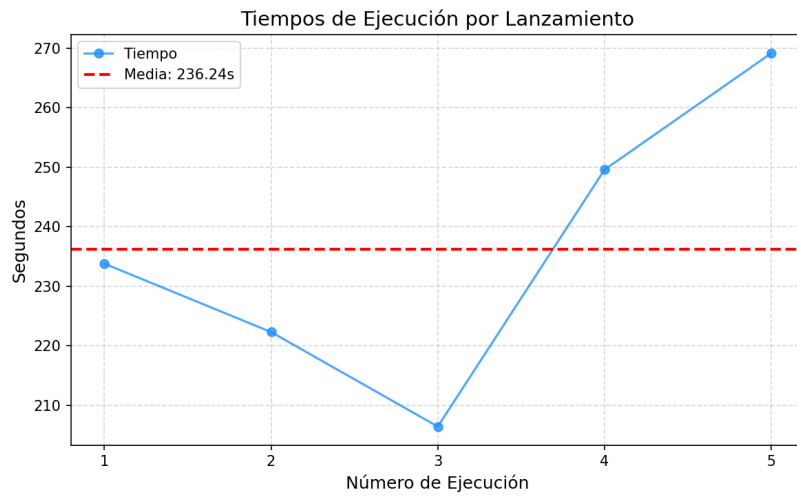


Figure 2: Distribución de tiempos de ejecución - Modelo A.

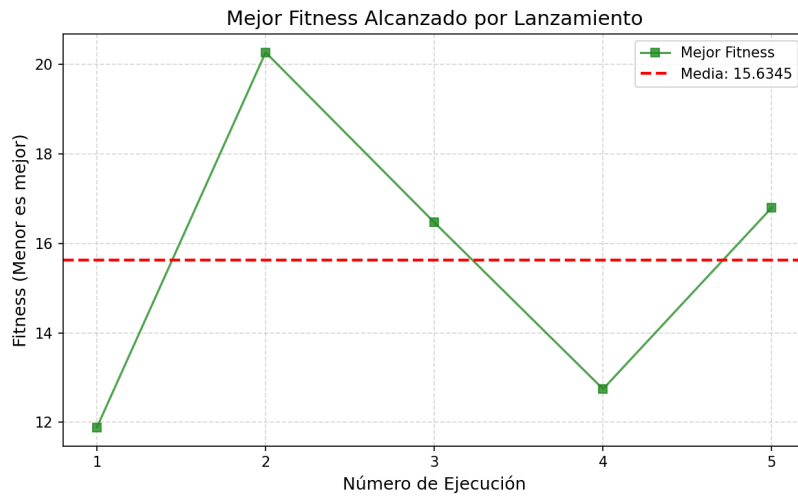


Figure 3: Evolución del mejor fitness por lanzamiento - Modelo A.

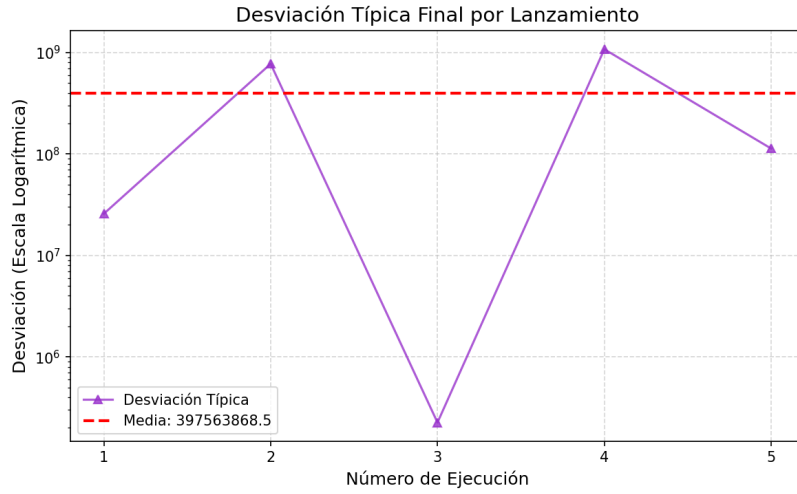


Figure 4: Desviación típica de la población - Modelo A.

12.2.2 Análisis del Modelo B (Círculo)

En el Modelo B, el espacio de búsqueda se restringió al rango $[-1, 1]$ con 15 puntos de bisección para capturar una geometría cerrada:

- **Muestreo y Precisión:** La reducción del área de búsqueda permite que el algoritmo profile la frontera circular con mayor fidelidad en menos tiempo (ver Fig. 5).
- **Estabilidad del Fitness:** El fitness medio (ver Fig. 6) tiende a ser más bajo y estable que en el modelo anterior, minimizando las "explosiones numéricas" gracias al rango acotado de las coordenadas.
- **Exploración Activa:** La desviación típica final (ver Fig. 7) muestra una población que mantiene capacidad de exploración activa, evitando la convergencia prematura en un único punto del círculo.

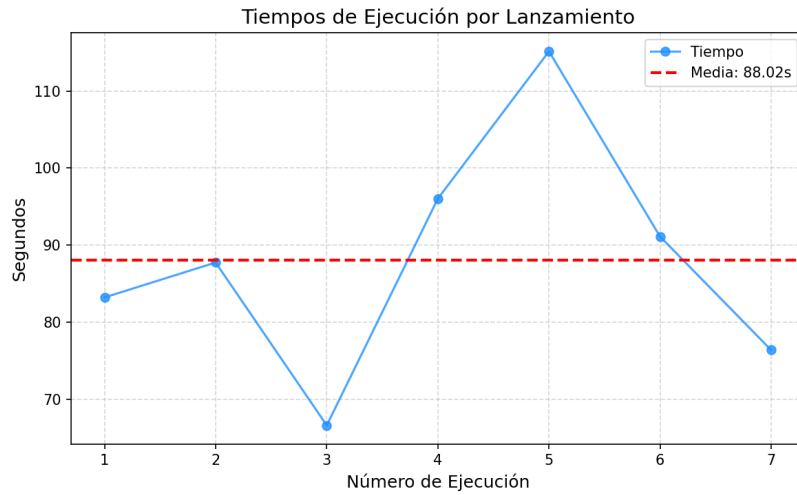


Figure 5: Distribución de tiempos de ejecución - Modelo B.

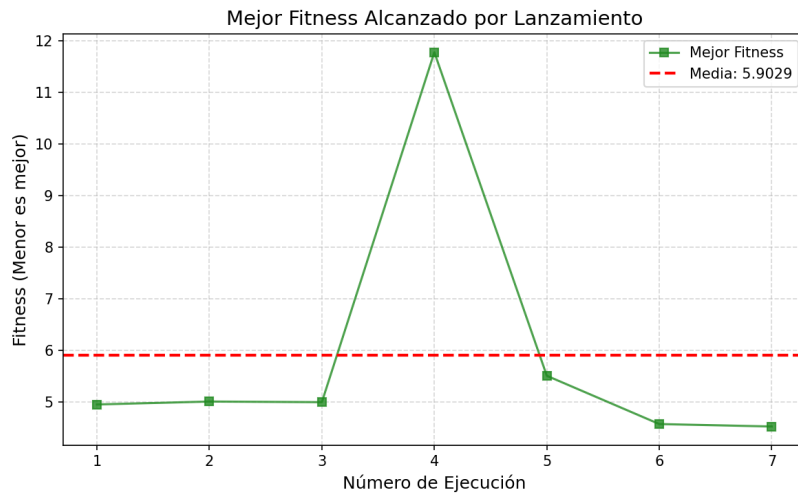


Figure 6: Evolución del mejor fitness por lanzamiento - Modelo B.

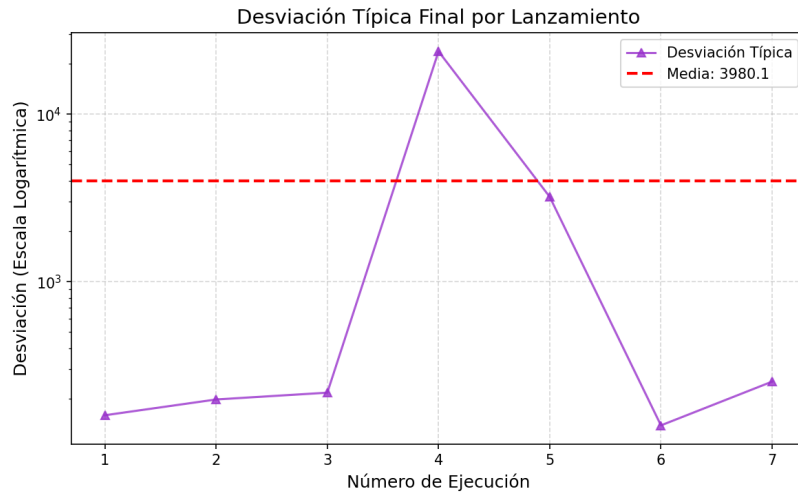


Figure 7: Desviación típica (Escala Logarítmica) - Modelo B.

Conclusión: La regresión simbólica demuestra ser eficaz para traducir cajas negras a ecuaciones explícitas. Mientras el Modelo A requiere mayor esfuerzo para capturar la apertura parabólica, el Modelo B se beneficia de un espacio restringido para cerrar la geometría del círculo con alta fidelidad.